ExamCell – Automated University Examination & Resource Scheduler

Java Mini-Project Presentation

Your Name

Department of Computer Engineering
Your University Name

February 17, 2026

Problem Statement

Objective:
To engineer a Java-based automation tool that generates conflict-free university exam timetables by intelligently mapping subjects to time slots, classrooms, and faculty supervisors.

Problem Overview

- **Complexity:** Manual scheduling is labor-intensive and error-prone.
- **Conflict Risk:** High probability of double-booking student batches or rooms.
- **Resource Allocation:** Inefficient distribution of faculty supervision duties.
- **Necessity:** A critical need for an algorithmic solution to enforce academic constraints.

Proposed Solution

We propose a **hybrid Java application** utilizing a **Backtracking Constraint Satisfaction Algorithm** to automate scheduling.

The system bridges the gap between desktop administration and web accessibility:

- **Admin Side:** Java Swing (Desktop)
- **User Side:** JSP & Servlets (Web)

System Architecture

1. **Input Layer**
   - Academic Data: Subjects (Dept/Sem) & Student Counts.
   - Infrastructure: Room Capacities & Faculty Lists.
2. **Processing Layer**
   - Algorithmic conflict detection (Time/Room/Faculty).
   - Recursive backtracking for optimal slot assignment.
3. **Output Layer**
   - Master Timetable generation.
   - Detailed allocation reports (Room & Faculty specific).

OOP Design Structure

- **Abstraction:** `Scheduler` (Abstract Base Class) defining the contract.
- **Implementation:** `BacktrackingScheduler` (Concrete Class) executing the logic.
- **Domain Models:** `Subject`, `StudentGroup`, `Classroom`, `Faculty`.
- **Utilities:** `DatabaseManager` (Persistence), `FileExporter` (Reporting).

Scheduling Engine Logic

1. **Ingest Data:** Load academic parameters and infrastructure limits.
2. **Selection:** Pick the next unscheduled subject (prioritized by difficulty/size).
3. **Assignment:** Tentatively assign a Time Slot and Room.
4. **Validation:** Check for Batch Overlap, Room Overflow, and Faculty Availability.
5. **Recursion:** If valid, proceed; if invalid, **Backtrack** and retry.
6. **Finalization:** Commit the conflict-free schedule to the database.

Java Concepts Used

- **Core OOP:** Encapsulation (Models), Inheritance (Schedulers), Polymorphism.
- **Advanced Logic:** Recursion (Backtracking), Collections (HashMaps for lookups).
- **Architecture:** JDBC (Data persistence), Multithreading (Background processing).
- **Hybrid UI:** Swing (Admin Control) + JSP/Servlets (Student View).

Conclusion & Future Scope

Conclusion

- Eliminates administrative bottlenecks and human error.
- Demonstrates practical application of Data Structures (Graphs/Trees) and Java development.

Future Enhancements

- Granular seating charts (Row/Seat specific).
- Genetic Algorithms for optimized gap scheduling.
- Automated email/SMS notifications.

Thank You!

Questions?